

Project 1: Intelligent Cloud-Based Log Analytics & BI System

Course: CSC11006 - Introduction to Cloud Computing

Weight: 15% of Total Grade

Duration: 2 Weeks

1. Project Overview

In this project, students will assume the role of a Cloud Data Engineer at a digital media startup. The company needs a scalable, serverless pipeline to ingest website traffic logs, analyze user sentiment in real-time using AI, and visualize business performance.

Students will build an end-to-end system on Google Cloud Platform (GCP) that ingests simulated data, processes it using Cloud Functions and Machine Learning APIs, stores it in BigQuery, and visualizes insights via Looker Studio.

2. Learning Objectives

By completing this project, students will be able to:

- **G2.1:** Architect a serverless data pipeline using Pub/Sub and Cloud Functions.
 - **G3.1:** Implement secure access controls using IAM (Service Accounts).
 - **G6.2:** Apply AI-as-a-Service by integrating the Cloud Natural Language API.
 - **G6.3:** perform predictive analytics using BigQuery ML (SQL-based Machine Learning).
-

3. System Architecture

The system follows an event-driven architecture:

1. Ingestion: A Python script simulates web traffic and user feedback, publishing events to Cloud Pub/Sub.
 2. Processing (ETL): A Cloud Function triggers on every message, calls the Natural Language API to score sentiment, and formats the data.
 3. Storage: Data is streamed into a BigQuery Data Warehouse.
 4. Analytics: BigQuery ML predicts user purchase behavior.
 5. Visualization: Looker Studio connects to BigQuery to display live dashboards.
-

4. Project Tasks & Requirements

Phase 1: Environment & Security Setup

- Create a GCP Project.
- Create a Service Account for the Cloud Function with restricted roles (Pub/Sub Subscriber, BigQuery Data Editor, Cloud Functions Invoker).
- Enable required APIs: [Pub/Sub](#), [Cloud Functions](#), [BigQuery](#), [Cloud Natural Language](#).

Phase 2: Data Ingestion (Pub/Sub)

- Create a Pub/Sub Topic named [website-logs](#).
- Run the Log Simulator Script (provided in Appendix A) to generate traffic with fields: [user_id](#), [action](#), [page](#), [response_time_ms](#), and [feedback](#).

Phase 3: Intelligent Processing (Cloud Functions + AI)

- Deploy a Cloud Function (Python/Node.js) triggered by the Pub/Sub topic.
- ML Requirement: The function must detect if the log contains [feedback](#). If yes, it must call the Natural Language API to generate a [sentiment_score](#) (-1.0 to 1.0).
- Insert the enriched record into BigQuery.

Phase 4: Predictive Analytics (BigQuery ML)

- Create a dataset [analytics_ds](#) and table [website_logs](#).
- ML Requirement: Write and execute a SQL query to train a Logistic Regression Model that predicts if a user will purchase based on their [sentiment_score](#) and [response_time_ms](#).

Phase 5: Visualization (Looker Studio)

- Connect Looker Studio to the BigQuery table.
- Create a dashboard with:
 - Gauge Chart: Average Sentiment Score (0 to 1).
 - Time Series: Traffic volume per hour.
 - Pie Chart: Distribution of User Actions.

5. Evaluation Rubric (Total: 100 Points)

Criteria	Requirements	Points
Architecture & Security	Correct setup of Pub/Sub, IAM Roles, and Least Privilege access.	20
Pipeline Functionality	Data flows successfully from Simulator → BQ. Logs are parsed correctly.	25

Machine Learning (AI)	Cloud Function successfully retrieves sentiment scores via API. BQML model is created and trainable.	25
Visualization	Dashboard works, auto-refreshes, and clearly visualizes the sentiment data.	20
Report	• Report clearly structured (Introduction, Architecture, Implementation, Results, Conclusion) • Clear explanation of architecture, pipeline, and ML model • Well-written, professional formatting, diagrams included • If you	10
Members Information & Assessment	Missing	-10
Late Submission Policy	If you need to submit the project after the deadline, you must send a formal email request explaining the reason for the delay. The email should clearly state your situation and must be sent before or at the time of submission. All approved late submissions will receive a 30% deduction from the total score of the assignment	
AI Tool Usage Policy	The use of AI tools is allowed for this project; however, students must clearly document how the AI tools were used. You are required to: 1. Provide the prompts used with the AI tool from the beginning to the end of your project.	

	2. Describe any issues, errors, or unexpected results encountered while following the AI-generated guidance. 3. Explain how you analyzed and resolved these problems, including any troubleshooting steps or adjustments you made. This documentation should demonstrate your understanding of the project and your ability to critically evaluate and work with AI-generated outputs.	
--	---	--

Appendix A: Python Log Simulator Script

```
import json
import time
import random
from datetime import datetime
from google.cloud import pubsub_v1

# CONFIGURATION
PROJECT_ID = "your-project-id" # TODO: Update this
TOPIC_ID = "website-logs"

publisher = pubsub_v1.PublisherClient()
topic_path = publisher.topic_path(PROJECT_ID, TOPIC_ID)

FEEDBACK_SAMPLES = [
    "I love the new interface!", "The checkout is confusing.",
    "Great selection.", "Website is too slow.",
    "Amazing experience!", "I cannot find the search button."
]

def run_simulator():
    print(f"Publishing to {topic_path}...")
    try:
        while True:
            # 30% chance of leaving feedback
            comment = random.choice(FEEDBACK_SAMPLES) if random.random() < 0.3
            else None

            data = {
                "timestamp": datetime.utcnow().isoformat() + "Z",
                "user_id": f"USER_{random.randint(1,50):03d}",
                "action": random.choice(["view", "click", "add_to_cart",
"purchase"]),
                "page": random.choice(["/home", "/cart", "/product"]),
                "response_time_ms": random.randint(50, 800),
                "feedback": comment
            }

            future = publisher.publish(topic_path,
json.dumps(data).encode("utf-8"))
            print(f"Sent: {data['user_id']}")
            time.sleep(1)
        except KeyboardInterrupt:
            print("Stopped.")

if __name__ == "__main__":
    run_simulator()
```

Appendix B: BigQuery ML SQL Reference

Training the Model:

```
CREATE OR REPLACE MODEL `analytics_ds.purchase_prediction`  
OPTIONS(  
  model_type='logistic_reg',  
  input_label_cols=['is_purchase']  
) AS  
  
SELECT  
  IF(action='purchase',1,0) AS is_purchase,  
  sentiment_score,  
  response_time_ms  
FROM `analytics_ds.website_logs`  
WHERE sentiment_score IS NOT NULL;
```